ГБПОУ «Колледж автоматизации и информационных технологий № 20»

Методические указания к производственной практике

ПП.03 · ПМ.03 Ревьюирование программных продуктов
Неделя 2: модули 3–4 · специальность 09.02.07 · группа ИСПО332

Составитель: Беличенко А. А., 2026

Содержание 1. Как пользоваться методичкой 2. Подготовка к неделе 2 3. Модуль 3. Теория: характеристики кода 4. Модуль 3. Ход работы 5. Модуль 4. Теория: общие настройки команды 6. Модуль 4. Ход работы 7. Оформление отчёта и сдача 8. Контрольные вопросы 9. Источники

1. Как пользоваться методичкой

Методичка объясняет, *как* выполнить задания недели 2; *что* и *к какому сроку* сдать — в Т3 на неделю 2. Как и на неделе 1, у каждого модуля сначала краткая теория (её спрашивают на защите), потом шаги по порядку.

Модули 3 и 4 связаны. В модуле 3 вы измеряете код «Библиотеки КАИТ» и меняете его характеристики в двух разных средах разработки — и обнаруживаете, что у каждой среды свои привычки: одна сохраняет файл с CRLF, другая форматирует по-своему, в итоге в `git diff` сотни строк «шума». В модуле 4 вы решаете эту проблему для всей команды: кладёте правила в репозиторий, чтобы у любого участника код форматировался и проверялся одинаково.

Условности те же: `ПРОЕКТ` — папка вашего форка `kait-library`, `PP03` — папка репозитория практики `pp03-ispo332`. Команды выполняются в терминале из папки проекта, если не сказано иное.

2. Подготовка к неделе 2

2.1 Обновить репозиторий практики и шаблоны

В репозиторий практики добавлены материалы недели 2. Обновите его и ещё раз скопируйте шаблоны — скрипт добавит папки `practice/m03` и `practice/m04`, ничего из уже сделанного не трогая:

```
cd PP03
git pull                        (если скачивали ZIP — скачайте заново)
```

Затем перетащите папку проекта на `PP03\tools\install.cmd` (или `powershell -ExecutionPolicy Bypass -File PP03\tools\install.ps1 ПРОЕКТ`).

2.2 Программы

Программа	Зачем	Где взять, проверка
Расширения VS Code: *PHP Intelephense*, *EditorConfig for VS Code*	форматирование и инспекция PHP, поддержка `.editorconfig`	вкладка Extensions (Ctrl+Shift+X)
Вторая среда разработки: **Apache NetBeans**	бесплатная IDE с поддержкой PHP	установщик Codelerity со встроенной Java: ссылка «Codelerity packages» на netbeans.apache.org → Download. При первом открытии PHP-проекта NetBeans предложит активировать поддержку PHP — согласитесь

Программа	Зачем	Где взять, проверка
или **PhpStorm**	платная IDE, есть 30-дневный пробный период и бесплатная лицензия для студентов	jetbrains.com/phpstorm
lizard	подсчёт цикломатической сложности	`python -m pip install lizard`, проверка: `python -m lizard --version`
PHP в командной строке	запуск линтера в М4	`php -v`. Если «не является командой» — запустите `PP03\tools\setup_kait.cmd` или используйте полный путь `C:\xampp\php\php.exe`

Вторая среда выбирается одна: NetBeans или PhpStorm. Ниже инструкции даны для обеих; кто пользуется другой IDE с поддержкой PHP (Eclipse PDT и т. п.) — согласуйте с руководителем.

2.3 В паре или самостоятельно

В модуле 4 нужно проверить, что ваши настройки действительно передаются другому человеку. Это можно сделать двумя способами — выбирайте сами:

- **В паре** — партнёр проверяет ваш проект на своём компьютере, а вы — его. Договоритесь с одногруппником и впишите его в `practice/README.md` (строка «Партнёр по ревью»).
- **Самостоятельно** — вы сами изображаете нового участника команды: чистый клон в отдельную папку, пустой профиль VS Code, отдельная база данных. Лучше — на другом компьютере класса.

Требования к результату одинаковые, оценка от выбора не зависит.

2.4 Если у вас свой проект

Задания модулей 3 и 4 одинаковы для всех; меняются только инструменты — под язык и стек проекта. Всё, что в методичке написано про PHP, NetBeans и PHP_CodeSniffer, замените строкой из таблицы. Выбор согласуйте с руководителем 09.11.

Среды разработки

Стек проекта	Вторая среда (кроме VS Code)	Расширения VS Code
PHP, Laravel	PhpStorm или Apache NetBeans	PHP Intelephense
JavaScript, TypeScript (React, Vue, Node)	WebStorm	ESLint, Prettier
Python (Django, Flask, FastAPI)	PyCharm	Python, Ruff

Стек проекта	Вторая среда (кроме VS Code)	Расширения VS Code
Java (Spring)	IntelliJ IDEA Community	Extension Pack for Java, Checkstyle
Kotlin, Android	Android Studio	Kotlin
C#, .NET	Visual Studio Community или Rider	C# Dev Kit

Линтеры (модуль 4)

Стек	Линтер и форматтер	Файл правил	Проверка → автоисправление
PHP, Laravel	PHP_CodeSniffer (лежит в `PP03/tools/php`) или Laravel Pint	`phpcs.xml` или `pint.json`	`php …\phpcs.phar` → `php …\phpcbf.phar`; или `vendor\bin\pint --test` → `vendor\bin\pint`
JavaScript, TypeScript	ESLint + Prettier	`eslint.config.js`, `.prettierrc`	`npx eslint .` → `npx eslint . --fix` и `npx prettier --write .`
Python	Ruff	`ruff.toml` или `pyproject.toml`	`ruff check .` → `ruff check --fix .` и `ruff format .`
Java	Checkstyle	`checkstyle.xml`	`gradlew checkstyleMain` (или плагин Maven) → форматирование IDE по `.editorconfig`
Kotlin	ktlint	`.editorconfig`	`ktlint` → `ktlint -F`
C#, .NET	`dotnet format` и анализаторы .NET	`.editorconfig`	`dotnet format --verify-no-changes` → `dotnet format`

Правила замены:

- **Вторая среда** — любая полноценная IDE для вашего стека, отличная от VS Code. Если две IDE (например, Visual Studio и Rider) недоступны, второй средой может быть та, в которой проект разрабатывался изначально.
- **Метрики** считает тот же `metrics.py`: lizard понимает PHP, JavaScript, TypeScript, Python, Java, Kotlin, C#, Swift и другие языки.
- **Табуляции и CRLF может не оказаться** — тогда запишите это как результат замера; шаг 4 М3 выполняется на тех изменениях, которые нужны вашему коду (переименование и выделение методов есть в любом проекте).
- **Линтер на чужом проекте** может найти сотни ошибок. Автоисправление — отдельным коммитом; то, что не исправляется автоматически, разберите по видам:

исправьте вручную или обоснованно отключите правило в файле правил. Остаток — с объяснением в `practice/m04/README.md`.

- **`.gitattributes` и `.editorconfig`** из `PP03/templates/week2/` подходят любому проекту; проверьте размер отступов (в JS/TS обычно 2 пробела, в Python, PHP, Java, C# — 4).
- **`.vscode/settings.json`** — замените секцию `[php]` на свой язык и форматтер; `extensions.json` — на расширения из таблицы.

3. Модуль 3. Теория: характеристики кода

3.1 Что измеряют в коде

Характеристика	Что показывает	Чем измерить
Объём (LOC)	сколько строк кода, комментариев и пустых строк	`metrics.py`
Цикломатическая сложность	сколько независимых путей выполнения у функции	lizard (через `metrics.py`), инспекции IDE
Длина функции	функцию длиннее 50 строк трудно удержать в голове	lizard
Замечания инспекции	неиспользуемые переменные, неопределённые функции, подозрительные конструкции	вкладка Problems в VS Code, инспекции второй IDE
Кодировка	в какой кодировке сохранены файлы (нужна UTF-8 без BOM)	строка состояния редактора
Окончания строк	LF (Linux, macOS) или CRLF (Windows)	`git ls-files --eol`, строка состояния
Отступы	пробелы или табуляция, сколько пробелов	строка состояния, `git grep`

Первые четыре характеристики описывают **качество** кода: насколько его легко понимать, тестировать и менять. Последние три — **оформление**: на работу программы не влияют, но если в команде они разные, каждый коммит тянет за собой изменения, не относящиеся к делу.

3.2 Цикломатическая сложность

Цикломатическая сложность (метрика Маккейба) — число независимых путей через функцию. Считается так: 1 + число точек ветвления (`if`, `elseif`, `case`, `for`, `foreach`, `while`, `catch`, тернарный оператор `?:`, логические `&&` и `||` в условиях).

```
function discount($sum, $isStudent)      // 1
{
    if ($sum > 5000) {                   // +1
        return 10;
    }
    if ($isStudent && $sum > 1000) {     // +1 за if, +1 за &&
        return 5;
    }
    return 0;
}                                         // итого 4
```

Чтобы проверить такую функцию полностью, нужно не меньше четырёх тестов. Принятые пороги: до 10 — нормально, 11–20 — функцию стоит разбить, больше 20 — функция почти наверняка содержит ошибки и её боятся менять.

3.3 Рефакторинг

Рефакторинг — изменение структуры кода **без изменения поведения**. Основные приёмы:

- **Rename (переименование)** — заменить невнятное имя (`$r`, `$b`, `$data2`) понятным во всех местах сразу.
- **Extract Method (выделение метода)** — вынести фрагмент функции в отдельный метод с понятным именем. Сложность исходной функции уменьшается на число ветвлений во вынесенном фрагменте.
- **Reformat** — привести отступы, пробелы и переносы к единому стилю.

Почему через IDE, а не вручную: IDE находит все использования имени, в том числе в других файлах, и не заденет одноимённую переменную в соседней функции; при выделении метода сама определяет, какие переменные передать параметрами и что вернуть.

> **Правило модуля.** Рефакторинг не исправляет ошибки. Если по ходу работы вы заметили ошибку в логике — запишите её в раздел «Находки» файла `metrics.md` и не исправляйте: ошибки исправляются в модуле 10, и там их нужно будет найти и доказать. Изменение, которое меняет поведение программы, в этом модуле считается ошибкой студента.

3.4 Кодировка и окончания строк

UTF-8 без BOM — стандартная кодировка исходного кода. BOM — три байта `EF BB BF` в начале файла. В PHP они выводятся в браузер раньше заголовков — и `header()`, `session_start()` перестают работать с ошибкой «headers already sent»; в shell-скриптах ломают первую строку `#!/bin/sh`, в JSON — делают файл невалидным для части программ.

LF и CRLF. Windows заканчивает строку двумя символами `\r\n` (CRLF), Linux и macOS — одним `\n` (LF). Git умеет преобразовывать их автоматически (`core.autocrlf`), но если

один участник сохранил файл с CRLF, а другой — с LF, git покажет изменённой каждую строку файла. Узнать, как хранятся файлы, можно командой:

```
git ls-files --eol
```

Колонка `i/` — как файл лежит в репозитории, `w/` — как в вашей рабочей папке. `i/lf w/crlf` — нормально для Windows с `core.autocrlf true`; `i/crlf` — в репозиторий попал файл с CRLF.

4. Модуль 3. Ход работы

Результат модуля — заполненный `practice/m03/metrics.md`, вывод скрипта `metrics-raw.md`, несколько коммитов с изменениями и отчёт.

Шаг 0. Ветка

```
git switch main
git pull
git switch -c practice/m03
```

Шаг 1. Проект в двух средах (1 ч)

VS Code: File → Open Folder → папка проекта. Установите расширения из п. 2.2. Внизу в строке состояния справа — кодировка (`UTF-8`), окончания строк (`CRLF`/`LF`), отступы (`Spaces: 4`), язык (`PHP`). Всё это кликабельно.

Для своего проекта — откройте его во второй среде из п. 2.4 обычным способом (File → Open) и пропустите два абзаца ниже.

NetBeans: File → New Project → PHP → *PHP Application with Existing Sources* → папка проекта; Run As — *Local Web Site*, Project URL — адрес, по которому проект открывается в браузере. NetBeans создаст в проекте папку `nbproject/` — пока не коммитьте её, это обсуждается в М4.

PhpStorm: File → Open → папка проекта. PhpStorm создаст папку `.idea/` — пока не коммитьте её.

Впишите версию второй среды в шапку `metrics.md`.

Шаг 2. Исходные метрики (2 ч)

1. Запустите подсчёт:

   ```
   python PP03\tools\metrics.py . --label "M03 до" --out practice\m03\metrics-raw.md
   ```

   Если в выводе написано «Сложность не посчитана» — не установлен lizard (п. 2.2).

2. Окончания строк и отступы:

```
git ls-files --eol
git grep -l -P "^\t" -- "*.php"
```

Первая команда покажет файлы с `i/crlf`, вторая — PHP-файлы, где строки начинаются с табуляции.

3. Инспекции. VS Code: вкладка Problems (Ctrl+Shift+M) — откройте по очереди все файлы `src/`, Intelephense проверяет открытые файлы. NetBeans: окно Window → Action Items (Ctrl+6), фильтр «текущий проект»; подсказки также видны значками на полях редактора. PhpStorm: Code → Inspect Code → Whole project.
4. Заполните столбец «M03 до» в `metrics.md` и сделайте скриншоты результатов инспекций.

Обратите внимание, какая функция самая сложная и где она находится. Сравните с горячими точками из паспорта проекта (M1): совпадение не случайно.

Шаг 3. Где что меняется (3 ч)

Для каждой строки таблицы «Где меняется каждая характеристика» найдите, как это делается в VS Code и во второй среде, и запишите путь в меню или сочетание клавиш. Подсказки:

Характеристика	VS Code	NetBeans	PhpStorm
Кодировка	строка состояния → `UTF-8` → Reopen / Save with Encoding	свойства проекта → Sources → Encoding (для всего проекта)	строка состояния или File → File Properties → File Encoding
Окончания строк	строка состояния → `CRLF` / `LF`	ищите сами — это вопрос для сравнения	строка состояния или File → File Properties → Line Separators
Табуляция → пробелы	строка состояния → `Tab Size` → Convert Indentation to Spaces	Tools → Options → Editor → Formatting → Expand Tabs to Spaces, затем форматирование	Edit → Convert Indents → To Spaces
Форматирование	Shift+Alt+F	Alt+Shift+F	Ctrl+Alt+L
Переименование	F2	Ctrl+R	Shift+F6
Выделение метода	ищите сами	ищите сами	Ctrl+Alt+M
Где хранятся настройки	`settings.json` пользователя или `.vscode/` проекта	Tools → Options; проект — `nbproject/`	Settings; проект — `.idea/`

Не каждая среда умеет всё. Бесплатная версия Intelephense, например, не делает переименование, а NetBeans — не каждое выделение метода для PHP. Если среда чего-то не умеет — так и запишите в таблицу: это и есть сравнение сред.

Шаг 4. Изменение характеристик (4 ч)

Выполните изменения ниже. Каждое — отдельным коммитом с понятным сообщением; в таблице «Что изменено» — какой средой и какой командой сделано. Хотя бы одно изменение должно быть сделано во второй среде.

1. **Отступы.** Найденный на шаге 2 файл с табуляцией переведите на пробелы. Коммит: `style: ReportService.php — отступы пробелами`.
2. **Окончания строк.** Файл с `i/crlf` пересохраните с LF. Проверьте: `git ls-files --eol` должен показать `i/lf`. Коммит: `style: окончания строк LF`.
3. **Переименование.** Найдите в `src/` 2–3 переменные с невнятными именами из одной буквы и переименуйте средствами IDE. Коммит: `refactor: понятные имена переменных в …`.
4. **Выделение методов.** Самую сложную функцию из шага 2 разбейте на части: вынесите в отдельные приватные методы законченные фрагменты (проверку состояния экземпляра, расчёт срока возврата, разбор очереди бронирований и т. п.). Цель — сложность самой функции не больше 10, длина не больше 50 строк. Каждое выделение — отдельный коммит: `refactor: выделен метод dueDate`.

Контрольный прогон. До начала шага 4 выполните в приложении 3–4 сценария, которые проходят через изменяемый код (выдача книги студенту и преподавателю, попытка выдать уже выданный экземпляр, выдача забронированной книги), и запишите, что ответила программа. После шага 4 повторите те же сценарии: ответы должны совпасть дословно. Запишите сценарии в `metrics.md`. Если ответ изменился — вы изменили поведение, откатите коммит (`git revert`).

Шаг 5. Конфликт форматирования (2 ч)

1. Возьмите один файл с кодом (в «Библиотеке КАИТ» — из `src/`). Отформатируйте его в VS Code (Shift+Alt+F), закоммитьте.
2. Откройте этот же файл во второй среде и отформатируйте её командой.
3. Посмотрите разницу:

   ```
   git diff --stat
   git diff
   ```

   Сделайте скриншот. Запишите в `metrics.md`, чем отличаются результаты (отступы, пробелы вокруг операторов, переносы, положение скобок, пустые строки).
4. Отмените форматирование второй среды: `git restore имя_файла`.

Если diff оказался пустым — повторите на файле посложнее: в «Библиотеке КАИТ» — из `templates/`, где PHP перемешан с HTML; в своём проекте — на шаблоне, компоненте с разметкой или файле с длинными строками. Объясните в отчёте, почему это проблема для команды: при ревью в Pull Request изменения логики теряются среди изменений оформления.

Шаг 6. Повторный замер, отчёт, сдача (3 ч)

```
python PP03\tools\metrics.py . --label "M03 после" --out practice\m03\metrics-raw.md
```

Заполните столбец «M03 после», раздел «Находки», отчёт `ISPO332_Familiya_M03.docx` по шаблону `report-templates/report-M03.docx`. Проверка и сдача — как в разделе 7.

5. Модуль 4. Теория: общие настройки команды

5.1 Три уровня настроек

Уровень	Где хранится	Пример	Попадает к другим участникам
Пользователь	профиль IDE на компьютере	тема, шрифт, сочетания клавиш	нет (только экспортом или синхронизацией аккаунта)
Проект	файлы в репозитории	`.editorconfig`, `.gitattributes`, `phpcs.xml`, `.vscode/settings.json`	да, вместе с `git clone`
Проверка	скрипт или CI	`php phpcs.phar` перед коммитом	да, и не зависит от IDE

Командное правило: всё, что влияет на содержимое файлов (кодировка, окончания строк, отступы, стиль), задаётся на уровне проекта и проверяется автоматически. Уровень пользователя — личное дело каждого.

5.2 Инструменты

- **`.editorconfig`** — простой текстовый файл с правилами для редакторов: кодировка, окончания строк, отступы, пустая строка в конце файла. Понимают PhpStorm и другие IDE JetBrains, Visual Studio, VS Code (через расширение) и десятки других редакторов. Apache NetBeans встроенной поддержки не имеет (старый сторонний плагин давно не обновляется) — поэтому для NetBeans правила приходится дублировать в настройках проекта, это хороший пример для сравнения способов. Не форматирует код — только задаёт базовые правила при сохранении.
- **`.gitattributes`** — правила для git: строка `* text=auto eol=lf` означает «в репозитории все текстовые файлы хранятся с LF». Работает у всех, кто клонирует репозиторий, независимо от их `core.autocrlf`.
- **Линтер** — программа, которая проверяет код по правилам стиля и находит подозрительные места. Для PHP — PHP_CodeSniffer: `phpcs` проверяет, `phpcbf` исправляет то, что можно исправить автоматически. Правила проекта лежат в `phpcs.xml`.
- **Стандарт PSR-12** — общепринятый стиль кода PHP: 4 пробела, LF, открывающая скобка класса и метода на новой строке, пробел после `if`, и т. д.

- **Форматтер** — инструмент, который переписывает оформление файла целиком (Shift+Alt+F). Линтер отвечает на вопрос «соответствует ли код правилам», форматтер — приводит код к правилам.
- **`.vscode/settings.json` и `extensions.json`** — настройки VS Code на уровне проекта и список рекомендуемых расширений.

5.3 Чего не класть в репозиторий

Личные и машинно-зависимые настройки: `nbproject/private/`, `.idea/workspace.xml`, пути к своим программам, пароли (`.env`), историю открытых файлов. Они мешают другим участникам и могут раскрыть секреты.

6. Модуль 4. Ход работы

Результат модуля — файлы командных настроек в корне проекта, заполненный `practice/m04/README.md`, экспорт настроек IDE, подтверждение проверки в чистом окружении и отчёт.

Готовые файлы-заготовки лежат в репозитории практики в папке `PP03/templates/week2/`.

Шаг 0. Ветка

```
git switch main
git pull
git switch -c practice/m04
```

Шаг 1. Окончания строк для всех: `.gitattributes` (1,5 ч)

Сначала закоммитьте всё, что есть, — шаг перезаписывает рабочую папку.

```
copy PP03\templates\week2\.gitattributes .
git add .gitattributes
git add --renormalize .
git status
git commit -m "chore: .gitattributes — окончания строк LF в репозитории"
git rm -r --cached -q .
git reset --hard
git ls-files --eol
```

`--renormalize` перезаписывает в репозитории файлы с CRLF; две следующие команды заново извлекают рабочую папку по новым правилам. В выводе последней команды у всех файлов должно быть `i/lf`. Запишите в отчёт, какие файлы изменила нормализация.

Шаг 2. `.editorconfig` (1 ч)

1. Скопируйте `PP03\templates\week2\.editorconfig` в корень проекта.
2. Прочитайте каждую строку и проверьте, что она соответствует проекту (отступы в PHP, JS и CSS — сколько пробелов на самом деле?). Поправьте при необходимости.

3. Проверка: в VS Code создайте файл `test.php`, поставьте отступ клавишей Tab, сохраните — в строке состояния должны быть `Spaces: 4`, `LF`, `UTF-8`. Повторите во второй среде и запишите, соблюдает ли она `.editorconfig` (PhpStorm — да; NetBeans — нет, правила для него задаются в шаге 4). Удалите `test.php`.
4. Коммит: `chore: .editorconfig`.

Шаг 3. Линтер PHP_CodeSniffer (3 ч)

Свой проект: возьмите линтер и файл правил из таблицы п. 2.4, команды проверки и автоисправления — оттуда же; порядок действий и коммиты — как ниже.

1. Скопируйте `PP03\templates\week2\phpcs.xml` в корень проекта. Прочитайте файл: что проверяется, какие папки и правила исключены и почему.
2. Запустите проверку из корня проекта (сам линтер лежит в репозитории практики, ставить ничего не нужно):

   ```
   php PP03\tools\php\phpcs.phar --report=summary
   php PP03\tools\php\phpcs.phar --report=source
   ```

   Первая команда — сколько ошибок в каких файлах, вторая — какие правила нарушаются чаще всего. Числа — в `practice/m04/README.md` и в строку «Ошибок линтера» в `practice/m03/metrics.md`.
3. Посмотрите подробности по одному файлу: `php PP03\tools\php\phpcs.phar src\ReportService.php`.
4. Автоисправление — отдельным коммитом, чтобы при ревью было видно, что логика не менялась:

   ```
   php PP03\tools\php\phpcbf.phar
   git diff --stat
   git commit -am "style: автоформатирование phpcbf по PSR-12"
   ```
5. Повторите проверку — ошибок должно стать 0. Откройте приложение и пройдите контрольные сценарии из М3: всё работает как раньше.
6. `phpcs.xml` — отдельный коммит: `chore: правила стиля PHP (phpcs.xml)`.

Шаг 4. Настройки IDE на уровне проекта и экспорт (3 ч)

VS Code — в репозиторий. Создайте в проекте папку `.vscode` и скопируйте туда `settings.json` и `extensions.json` из `PP03\templates\week2\vscode\`. Если в `.vscode` уже есть `launch.json` (от `setup_kait`), не удаляйте его, но проверьте: путь к PHP в нём личный, на другом компьютере он другой. Решите, коммитить ли `launch.json`, и обоснуйте в README. Коммит: `chore: настройки VS Code для проекта`.

VS Code — экспорт профиля. Шестерёнка (слева внизу) → Profiles → New Profile «ПП03» (на основе текущего) → … → Export → Export to file → сохранить в

`practice/m04/vscode.code-profile`.

Свой проект: настройте стиль кода второй среды на уровне проекта и экспортируйте её настройки: в IDE JetBrains — как для PhpStorm ниже, в Visual Studio — Tools → Import and Export Settings → Export, в Android Studio — File → Manage IDE Settings → Export Settings.

NetBeans. Свойства проекта (правый клик по проекту → Properties) → Formatting → *Use project specific options*: отступ 4 пробела, Expand Tabs to Spaces. Эти настройки сохраняются в `nbproject/project.properties` — его можно коммитить, а `nbproject/private/` — нет (добавьте в `.gitignore`). Полный экспорт настроек IDE: Tools → Options → кнопка Export внизу слева → `practice/m04/netbeans-settings.zip`.

PhpStorm. Settings → Editor → Code Style → Scheme: *Project* → Set from… → PSR12. Настройки сохраняются в `.idea/codeStyles/` — эту папку можно коммитить, остальное в `.idea/` — нет (добавьте исключения в `.gitignore`). Полный экспорт: File → Manage IDE Settings → Export Settings → `practice/m04/phpstorm-settings.zip`.

Шаг 5. CONTRIBUTING.md (2 ч)

Скопируйте `PP03\templates\week2\CONTRIBUTING.md` в корень проекта и заполните все разделы: что установить, как запустить проект с нуля (с базой данных и тестовыми пользователями), какие настройки редактора есть в проекте, как проверять стиль, как называть ветки и коммиты, чего не коммитить. Пишите для человека, который видит проект впервые. Коммит: `docs: CONTRIBUTING.md`.

Шаг 6. Проверка в чистом окружении (3 ч)

Отправьте ветку на GitHub: `git push -u origin practice/m04`. Дальше — один из двух вариантов (п. 2.3).

Вариант А — в паре. Дайте партнёру ссылку на ветку. Партнёр:

1. Засекает время, клонирует ваш форк в новую папку (`git clone ссылка kait-check`), переходит в неё и переключается на ветку: `git switch practice/m04`.
2. Открывает проект в VS Code, соглашается установить рекомендованные расширения.
3. Запускает проект и линтер, читая только ваш `CONTRIBUTING.md`.
4. Форматирует 3–5 файлов с кодом (Shift+Alt+F) и сохраняет, затем выполняет `git status` и `git diff`.

Затем то же самое вы делаете для партнёра.

Вариант Б — самостоятельно. Вы проверяете свой проект так, как будто видите его впервые:

1. Если можно — сядьте за другой компьютер класса. Если нет — создайте в VS Code пустой профиль: шестерёнка → Profiles → New Profile → «Проверка», *без копирования* настроек и расширений. В нём VS Code ведёт себя как только что установленный.

2. Засеките время и клонируйте форк в новую папку рядом с рабочей: `git clone ссылка kait-check`, затем `cd kait-check` и `git switch practice/m04`.
3. Откройте `kait-check` в профиле «Проверка» и соглашайтесь только на то, что предлагает сам проект (рекомендованные расширения). Ничего не настраивайте вручную.
4. Разверните проект, читая только свой `CONTRIBUTING.md`, с отдельной базой данных (например, `kait_check`), чтобы не опираться на уже настроенную. Каждое место, где пришлось вспоминать или догадываться, — недостаток инструкции: исправьте `CONTRIBUTING.md` в рабочей папке.
5. Запустите линтер, отформатируйте 3–5 файлов с кодом, выполните `git status` и `git diff`.

В обоих вариантах ожидаемый результат — **пустой diff**: настройки передались, форматирование в чистом окружении совпадает с вашим. Скриншот — `practice/m04/check-empty-diff.png`. Если diff не пустой — выясните, какая настройка не передалась, исправьте файлы в репозитории и повторите. Найденная причина — хороший материал для отчёта. Результаты — в таблицу «Проверка в чистом окружении» в `practice/m04/README.md`; после проверки папку `kait-check` и профиль «Проверка» можно удалить.

Шаг 7. Сравнение способов, отчёт, сдача (2,5 ч)

Заполните таблицу сравнения способов передачи настроек в `practice/m04/README.md` и сделайте вывод: какой способ подходит для команды и почему. Отчёт `ISPO332_Familiya_M04.docx` — по шаблону `report-templates/report-M04.docx`.

7. Оформление отчёта и сдача

Оформление отчёта — как на неделе 1: шаблоны `report-templates/report-M03.docx` и `report-M04.docx`, сохранить как `ISPO332_Familiya_M03.docx` и `…_M04.docx`, заменить серые подсказки своим текстом, обновить оглавление.

В разделе «Ход работы» для М3 обязательно: таблица метрик «до / после», таблица сравнения сред, скриншоты инспекций и `git diff` конфликта форматирования, перечень коммитов с изменениями. Для М4: содержимое и назначение каждого добавленного файла настроек, числа линтера до и после, скриншот пустого diff в чистом окружении, таблица сравнения способов.

Сдача модуля:

1. Отчёт — в `practice/mNN/`, коммит.
2. Проверка: `python PP03\tools\check_practice.py . --module 3` (для М4 — `--module 4`) — без ошибок.
3. Pull Request из `practice/m03` (`practice/m04`) в `main`, заполнить описание, влить.

4. Отметить модуль `[x]` в `practice/README.md`.

8. Контрольные вопросы

Модуль 3

1. Что такое цикломатическая сложность? Посчитайте её для любой функции вашего проекта.
2. Какая функция проекта самая сложная, почему именно она и как вы её упростили?
3. Чем рефакторинг отличается от исправления ошибок? Почему в модуле 3 нельзя исправлять найденные ошибки?
4. Почему переименование и выделение метода надёжнее делать средствами IDE, а не вручную?
5. Чем LF отличается от CRLF? Что показывает `git ls-files --eol`?
6. Чем опасен BOM в начале файла с кодом? Приведите пример для языка вашего проекта.
7. Какие различия между VS Code и второй средой вы обнаружили? Какая среда удобнее для этого проекта и почему?

Модуль 4

1. Чем `.editorconfig` отличается от настроек конкретной IDE?
2. Зачем нужен `.gitattributes`, если у каждого настроен `core.autocrlf`?
3. Чем линтер отличается от форматтера? Приведите пример правила PSR-12.
4. Почему автоформатирование коммитят отдельно от изменений логики?
5. Какие настройки IDE можно класть в репозиторий, а какие нельзя? Приведите примеры из своего проекта.
6. Сравните передачу настроек через репозиторий, экспорт профиля и синхронизацию аккаунта.
7. Что не передалось в чистое окружение с первого раза и почему?

9. Источники

1. McCabe T. J. A Complexity Measure // IEEE Transactions on Software Engineering. — 1976. — № 4. — С. 308–320.
2. Фаулер М. Рефакторинг: улучшение проекта существующего кода. 2-е изд. — СПб.: Диалектика, 2019.
3. PSR-12: Extended Coding Style. — URL: https://www.php-fig.org/psr/psr-12/
4. PHP_CodeSniffer. — URL: https://github.com/PHPCSStandards/PHP_CodeSniffer
5. EditorConfig. — URL: https://editorconfig.org

6. Git: gitattributes — URL: https://git-scm.com/docs/gitattributes ; Pro Git, раздел 8.1 «Конфигурация Git». — URL: https://git-scm.com/book/ru/v2
7. Visual Studio Code: Profiles; Workspace settings. — URL: https://code.visualstudio.com/docs
8. lizard — анализатор цикломатической сложности. — URL: https://github.com/terryyin/lizard